Release Management

& GO-LIVE STRATEGY DOCUMENT

Process Definition & Procedures

1. PURPOSE

This document formally defines the release management process, branching strategy, code freeze procedures, deployment pipelines, hotfix management, versioning strategy, and rollback procedures for the application. It ensures a standardized, secure, and highly available go-live workflow across all environments.

2. APPLICATION LANDSCAPE

Component Type	Technologies & Services
UI Application	ASP.NET MVC Application Azure App Service
API Application	ASP.NET Web API Azure App Service
Database	Azure SQL Database DACPAC Deployment
Security & Auth	Legacy Authentication Azure Entra ID SSO (Future Release)
Supporting Services	Azure Key Vault, Azure DevOps, Power Automate (Email Notifications), Microsoft Graph API

3. BRANCHING STRATEGY

The application strictly follows an environment-based branching model to ensure separation of concerns and controlled promotion of code.

```
feature/* → dev → uat → preprod → prod
```

Branch Name	Purpose & Environment
`feature/*`	Individual feature development, bug fixes, and targeted enhancements.
`dev`	Integration environment for compiling and testing merged features.
`uat`	Dedicated branch for Business testing and User Acceptance.
`preprod`	Final validation, staging environment matching production configurations.
`prod`	Production releases. Reflects the current live state of the application.

4. PULL REQUEST (PR) STRATEGY

Feature Development Flow

Developers create feature branches from the latest `dev` branch. Branch names must be descriptive:
Examples: `feature/entraid-sso`, `feature/report-enhancement`, `feature/customer-import`

Merge Pipeline

Direct commits are strictly prohibited in `uat`, `preprod`, and `prod` branches. Code is promoted exclusively via Pull Requests in the following sequence:

- `feature/*` → `dev`
- `dev` → `uat`
- `uat` → `preprod`
- `preprod` → `prod`

5. BRANCH POLICIES & PROTECTION

Branch	PR Requirements & Approvals
Dev	• Pull Request Required • Minimum 1 Reviewer • Successful Build Validation
UAT	• Pull Request Required • Minimum 2 Reviewers • Successful Build Validation • Comment Resolution Required
PreProd	• Pull Request Required • Minimum 2 Reviewers • Successful Build Validation • Change Approval Required
Prod	• Pull Request Required • Minimum 2 Reviewers • Successful Build Validation • Release Manager Approval Required • **No Direct Push Allowed**

6. FREEZE PROCEDURES

Feature Freeze Process

Objective: Prevent new features from destabilizing release candidate builds.

Trigger: Commences immediately after *UAT Sign-off* OR *one week before* production go-live.

Allowed Changes

- Critical Defects
- Security Fixes
- Deployment Fixes

Restricted Changes

- New Features / UI Enhancements
- Refactoring / DB Design Changes
- Entra ID SSO Enhancements

Code Freeze Process

Objective: Absolute stabilization of the production release candidate.

Trigger: Commences after *PreProd Approval* and *Go-Live Readiness Review*.

Allowed Changes (RM Approval Required)

- P1 Production Defects
- Deployment/Pipeline Issues

Note: Any changes introduced during Code Freeze require explicit, documented approval from the designated Release Manager.

7. BRANCH LOCK PROCEDURE

The `prod` branch is physically locked in Azure DevOps to prevent merge operations during critical phases.

When to Lock:

- During Production Deployments

- During Release Validation windows
- During Emergency Production Fix implementation

Azure DevOps Lock Steps:

`Repos → Branches → Select 'prod' → More Options (...) → Lock`

Unlock Condition: Unlock only after *Successful Production Validation* and final *Release Sign-Off*.

8. VERSIONING & TAGGING STRATEGY

Semantic Versioning (MAJOR.MINOR.PATCH)

The application follows standard semantic versioning (e.g., `1.0.0`).

- **Major (X.0.0):** Breaking changes, architecture redesign, or major platform migrations.
- **Minor (0.X.0):** New functionality (e.g., Entra ID SSO Release, New Reporting Module).
- **Patch (0.0.X):** Bug fixes, UI corrections, and API issue resolution.

Release Scenario	Version Example
Initial Go Live	1.0.0
Production Hotfix	1.0.1
Additional Hotfix	1.0.2
Entra ID SSO Release	1.1.0
Major Upgrade	2.0.0

Git Tagging Strategy

Tags provide immutable release snapshots, ensuring release traceability, rollback support, audit compliance, and deployment verification.

Tag Creation Execution:

```
git checkout prod
git pull
git tag -a v1.0.0 -m "Production Release 1.0.0"
git push origin v1.0.0
```

9. GO-LIVE PROCESS PIPELINE

Step 1: Business Sign-Off

Formal approval from Product Owner & Product Manager.

Step 2: UAT Sign-Off

Approval from Business Team and Business Analyst (BA).

Step 3: Feature Freeze

Implementation of freeze. No new features permitted.

Step 4: PreProd Validation

Comprehensive validation of UI, API, Database, Integrations, Key Vault, Power Automate, and Security.

Step 5: Code Freeze

Release Candidate is approved and locked.

Step 6: Production Deployment

Execute deployment in order: **(1)** Database DACPAC → **(2)** API → **(3)** UI → **(4)** Smoke Testing.

Step 7: Production Validation

Validate Login, User Roles, Key Business Flows, Email Notifications, API & Database Connectivity.

Step 8: Release Tag Creation

Generate immutable Git tag (e.g., `v1.0.0`).

Step 9: Production Sign-Off

Final acknowledgment by Product Owner, Business Owner, and Project Manager.

10. ENTRA ID SSO RELEASE STRATEGY

The deployment of Azure Entra ID Single Sign-On is separated across phases to mitigate risk.

> **Phase-I Current State**
>
> Legacy Authentication is Enabled. Entra ID is explicitly Disabled.

Development & Toggle Strategy

SSO changes must remain isolated in `feature/entraid-sso` and cannot merge to the production branch until explicitly approved for Phase-II.

Feature Toggle Configuration:

```
{
  "Features": {
    "EnableSSO": false  // Set to true in Future Release
  }
}
```

11. INCIDENT MANAGEMENT: HOTFIX & ROLLBACK

Hotfix Process

When a production issue is identified:

1. Create branch `hotfix/issue-name` directly from `prod`.
2. Deploy fix directly to Production.
3. Create new patch tag (e.g., `v1.0.1`).
4. Backport fix via PRs: `prod` → `preprod` → `uat` → `dev`.

Rollback Strategy

Triggers: Critical production failures, deployment failures, data corruption, or security issues.

- **Application:** Redeploy the previous successful Git tagged build (e.g., fallback to `v1.0.0`).
- **Database:** Execute rollback scripts, perform Backup Restore, or initiate Azure SQL Point-in-Time Restore.

12. ROLES & RESPONSIBILITIES

Role	Primary Responsibility
Developer	Feature Development & Bug Fixing
Lead Developer	Code Review & Technical Oversight
Business Analyst (BA)	Requirement Validation & UAT Coordination
Product Owner	Business Approval & Feature Prioritization
Product Manager	Release Approval & Strategy Alignment
Release Manager	Deployment Coordination & Environment Locking

Project Owner	Final Go-Live Authorization

13. GO-LIVE READINESS CHECKLIST

User Interface & API Validations	Database, Security & Integrations
☐ Application Starts successfully ☐ Login routing works correctly ☐ Application Navigation operates normally ☐ No Console Errors present **API Checks** ☐ Health Check Endpoints Pass ☐ Authentication & Authorization Active ☐ API Response Validation passed	**Database Validations** ☐ DACPAC Deployment Successful ☐ Post-Deployment Scripts Executed ☐ Data Sanity/Validation Complete **Security & Operations** ☐ Key Vault Access / Secrets Loaded ☐ Certificates are Valid ☐ SSO Disabled (Phase-I verified) ☐ Power Automate / Graph API Working ☐ Release Notes Published & Tag Created